```
template<typename T>
class lock_free_queue
{
private:
    struct node
    {
        void release_ref()
        {
            node_counter old_counter=
                count.load(std::memory_order_relaxed);
            node_counter new_counter;
            do
            {
                new_counter=old_counter;
                --new_counter.internal_count;     <-- (1)
            }
            while(!count.compare_exchange_strong(     <-- (2)
                  old_counter,new_counter,
                  std::memory_order_acquire,std::memory_order_relaxed));

            if(!new_counter.internal_count &&
               !new_counter.external_counters)
            {
                delete this;     <-- (3)
            }
        }
    };
};
```

The implementation of `node::release_ref()` is only slightly changed from the equivalent code in the implementation of `lock_free_stack::pop()` from listing 7.11. Whereas the code in listing 7.11 only has to handle a single external count, so you could just use a simple `fetch_sub`, the whole `count` structure now has to be updated atomically, even though you only want to modify the `internal_count` field ❶. This therefore requires a compare/exchange loop ❷. Once you've decremented the `internal_count`, if both the internal and external counts are now zero, this is the last reference, so you can delete the node ❸.

Listing 7.18. Obtaining a new reference to a node in a lock-free queue

```
template<typename T>
class lock_free_queue
{
private:
```